2026 Linux C/C++ 高性能项目路线图 (Route C)

> **版本说明**：V23.9 **Absolute Final Sealed Edition（终极封板面试版）**
> **核心战略**：**MVP First, Pro Later**
> **执行原则**：通过明确边界与工程约束 **降低** 死锁 / OOM / UAF 风险（best-effort）；所有优化必须有**可复现**的 Benchmark 数据支撑。

☑ 统一 Benchmark 口径（全项目通用）

- **构建环境**：Linux x86_64；GCC >= 12；`-O2 -DNDEBUG`；关闭 ASan/TSan（除非专门跑 Sanitizer 口径）。
- **压测环境**：单机压测优先 `127.0.0.1` loopback；记录 CPU 型号/核心数/内存/内核版本。
- **计时标准**：所有吞吐/延迟统计必须写清：**Warmup（预热）时长**、测量时长、样本量、P99 计算方式。
- **P50/P95/P99 统一算法**：对测量窗口内全部延迟样本排序，按 nearest-rank 取 `ceil(p * N)` 项（p=0.50/0.95/0.99）。
- **空样本口径**：若测量窗口内 `N==0`，则该轮延迟指标无效，不计算 P50/P95/P99，不参与验收。
- **统计原子性**：计数器使用 `std::atomic`；说明每个指标的分子/分母各自“何时递增”。
  - **网关复用率**：`accept_count` 在 accept 成功时递增；`requests_total` 在成功解析完整请求时递增。复用率定义为 `requests_total / accept_count`（按测量窗口差分值计算）。复用率按验收用例的测量窗口起止差分统计（由用例定义 warmup/测量窗口），不在统一口径里固定 warmup 基线时刻；`accept_count==0` 记为无效（该轮不参与验收）。

📊 统一 Metrics 最小集（全项目通用）

> 目的：把“我做过”变成“我能证明”。所有项目至少输出一套最小可观测性指标，便于压测定位与面试复述。

- **输出频率（写死）**：每 1s 输出一行，写入 `results/.../metrics.log`。
- **统一格式（写死）**：`ts=<unix_ms> key=value key=value ...`（便于 grep/解析）。
- **全项目必含字段（写死）**
  - `uptime_s`：进程启动到当前的秒数（整数）。
  - `rss_kb`：`/proc/<pid>/status` 的 `VmRSS`（KB）。
  - `errors_total`：错误总数（可再按原因拆分 bucket）。
- **项目特有字段（最低要求）**
  - **日志**：`enqueue_total`、`enqueue_ok_total`、`dropped_newest_total`、`dropped_oldest_total`、`dropped_stop_total`、`queue_bytes_current`
  - **网关**：`accept_total`、`conns_current`、`requests_total`、`reject_total`（可按 431/413/411/501/503/429 分桶）、`global_inflight_bytes_current`
  - **IM**：`sessions_current`、`msg_in_total`、`msg_out_total`、`stale_session_drop_total`、`outbuf_overflow_total`、`offline_drop_total`

🩹 自测与证据口径（全项目通用，面试可复述）

- **三套构建口径（写死）**
  - `release`：`-O2 -DNDEBUG`
  - `asan_ubsan`：`-O1 -g -fsanitize=address,undefined`
  - `tsan`：`-O1 -g -fsanitize=thread`
- **自测分层（写死）**
  1. **单测/接口语义**：`ctest` 全绿，覆盖“失败语义/边界拒绝/计数递增时机”。
  2. **压力稳定性**：每个核心压测用例跑 **3 轮**，取**中位数**作为结论；同时保留 **raw latency samples**（用于复现 P99）。 若出现无效轮则丢弃并补跑至 3 轮有效后再取中位数；不足 3 轮有效则该用例整体无效、不参与验收。
  3. **内存安全**：`asan_ubsan` 口径下跑同等压力 **≥5 分钟**，要求无 UAF/越界/泄漏告警。
  4. **并发安全**：`tsan` 口径下跑关键并发路径用例（日志并发入队/网关关闭释放/IM 会话切换），要求无 data-race 告警。
- **证据落盘格式（写死）**：`results/<project>/<yyyymmdd>_<machine>/`
  - `machine.txt`（`uname -a`、`lscpu`、内存、内核版本）
  - `build_flags.txt`（CXXFLAGS/LDFLAGS 与 sanitizer 口径）
  - `bench_cmd.txt`（完整命令行与参数）
  - `latency_samples.txt`（延迟原始样本，便于复算 nearest-rank P99）
  - `summary.md`（一页摘要：p50/p95/p99/QPS/CPU/RSS 增量；QPS=测量窗口内完成一次处理闭环（成功解析完整请求/消息并完成写回/投递）的次数/窗口秒数（按差分），分子在闭环完成时 +1，被拒绝/超时/主动 close 不计入；日志项目不填 QPS，改填 throughput_mb_per_sec；分子=测量窗内成功入队 payload_bytes 差分，分母=窗口秒数。CPU=测量窗口内进程 CPU%(user+sys) 平均值（如 pidstat 1s 采样取均值）；RSS 增量=以 `/proc/<pid>/status` 的 `VmRSS` 为准，取 warmup 结束与测量结束差分（按差分））

🟢 阶段一：MVP 基础落地（活下来）

目标：构建代码逻辑闭环、资源管理规范（RAII）、边界清晰的可用系统。

---

1) Project 1：基础异步日志库（MVP）

功能定义：实现一个多线程安全的异步日志系统。

核心接口（API）

- `Init()`：启动后台线程。
- `Log()`：并发边界防御
  - **防内存爆**：默认 `max_queue_bytes = 512MB`（写死默认值，可配置）。
  - **容量判定与一致性**：`current_bytes` 的 check/add/sub 与队列 push/pop 必须在同一把 queue mutex 临界区内完成，使 `max_queue_bytes` 成为强约束（多生产者下不超额入队）。
  - **溢出语义（收口）**：
    - `DROP_NEWEST`（默认）：若 `entry_cost > max_queue_bytes - current_bytes`（要求临界区内 `current_bytes<=max_queue_bytes`）→ **拒绝当前条目**，`Log()` 返回 `false`，并递增 `dropped_newest_total`。

    - `DROP_OLDEST`：若超限 → 在**同一临界区**内弹出队首（同步 `current_bytes`）直到 `entry_cost <= max_queue_bytes - current_bytes`；**每弹出 1 条队首递增** `dropped_oldest_total`；若仍无法容纳而拒绝当前条目，则递增 `dropped_newest_total`。
  - **Stop 收敛**：若 `m_stop == true`，`Log()` **直接拒绝入队**并递增 `dropped_stop_total`。
  - **统一记账口径（写死）**：`entry_cost = msg_len + 128B`（计算前先判 `max_queue_bytes<=128B` 或 `msg_len > max_queue_bytes - 128B` 直接拒绝并计入 `dropped_newest_total`）；`current_bytes` 统计 `entry_cost` 之和；所有容量判定与 add/sub 一律使用 `entry_cost`，用于**降低** OOM 风险（best-effort）；`max_queue_bytes` 约束的是该记账值，进程 RSS 可能略高。`Log()` 必须在做任何 payload 拷贝/分配前先完成上述 guard，并在持锁临界区内用 `entry_cost > max_queue_bytes - current_bytes`（要求 `current_bytes<=max_queue_bytes`）完成超限判定；若 `entry_cost > max_queue_bytes` 则直接拒绝当前条目返回 `false`（计入 `dropped_newest_total`）。
  - **接口语义**：返回 `bool`；`false` 表示拒绝入队（可能因为 stop/超限/drop）。
- `Stop()`：安全退出时序
  - 建议/期望在调用 `Stop()` 前逐步收敛生产者；`Stop` 置位后 `Log()` 将返回 `false` 并拒绝入队（可能丢弃），用于并发收敛。
  - **并发收敛（写死）**：`m_stop` 为 atomic；`Log()` 在持有队列 mutex 后二次检查 `m_stop` 再入队；`Stop()` 持同一把 mutex 置位 `m_stop=true` 后 `notify_all`，使置位与入队互斥。
  - 时序：置位 `m_stop` → `notify_all` → 后台线程 drain 队列至空 → `flush` → `join`。

技术实现

- **并发**：`mutex + condition_variable + deque`。
- **落盘策略**：每 **3s** 或累计 **1000 条**触发一次 `flush`。
- **持久性口径**：支持配置 `flush_mode`
  - `flush_only`（默认）：仅写入内核缓冲，高性能但断电丢数据。
  - `fsync_every_k` / `fsync_periodic`：定期调用 `fsync` 刷盘，提升持久性但降低吞吐。**参数写死**：`fsync_every_k` 每成功写入 `k` 条触发一次 `fsync`（默认 `k=1000`）；`fsync_periodic` 每 `period` 触发一次 `fsync`（默认 `period=3s`）。

验收标准

1. **完整性**：10 线程各写 10k 条，`wc -l` 严格等于 100k（前提：单条 msg 不含换行）。
   - **验收步骤**：生产者 join 后必须调用 `Stop()` 并等待后台线程 drain+flush 完成，再执行 `wc -l`。
   - **前提**：该用例固定单条 msg=256B（不含换行），并要求 max_queue_bytes>=64MB 且验收时 dropped_*==0；否则该轮无效不参与验收。
2. **吞吐量**：`Log()` 入队吞吐 > 20MB/s。
   - **口径声明**：按成功入队 payload 字节统计（与 dropped 分开），并声明该阈值在 `dropped=0` 的用例下校验。
   - **验收口径**：该阈值以 `flush_mode=flush_only` + **本机 SSD/NVMe 的本地文件系统目录**（非网络盘/非 tmpfs；不满足则该轮无效，不参与验收）为口径；跨机器仅记录结果不做硬阈值比较。
   - **关键指标**：记录 `enqueue_ns_p99`（入队延迟）与 `write_bytes_per_sec`（写调用字节率），用于定位瓶颈（锁 vs IO）。

    - **口径（写死）**：`enqueue_ns_p99`=测量窗口内单次 `Log()` 从进入函数到返回（成功入队或拒绝）耗时取 P99；`write_bytes_per_sec`=测量窗口内成功 `write/fwrite` 返回字节总和 / 窗口秒数（不宣称物理落盘带宽）。
  - **Benchmark 参数**：warmup **1s**，测量 **10s**；单条日志 **256B**；并发 **10 线程**。

☑ DoD（MVP 封板交付清单）

- ☐ `make release && ./scripts/run_demo.sh` 一键启动并写入日志文件（含停止 `Stop()`）。
- ☐ `make asan_ubsan` 口径下跑核心压力用例 ≥5 分钟，无 UAF/越界/泄漏告警。
- ☐ `make tsan` 口径下跑并发入队/Stop 收敛路径，无 data-race 告警。
- ☐ 至少 1 份可复现证据落盘：`results/log/<yyyymmdd>_<machine>/`（machine/build_flags/bench_cmd/latency_samples/summary.md）。
- ☐ README 说明：线程模型、`max_queue_bytes`/drop 语义、Stop 时序与复现步骤。

---

2) Project 2：基础 TCP 网关（MVP）

功能定义：基于 Epoll 的 TCP 服务器，解析 HTTP/1.1 GET/POST。

- **线程模型（写死）**：MVP 为 **单 Reactor 单 EventLoop 线程**（accept/IO 同线程）；`global_inflight_bytes`、`ip_token_bucket`、`ReadyQueue`、`pending_close_queue` 等全局结构由该线程串行维护（跨线程仅允许原子计数）。

技术实现

- **IO 模型**：`Epoll LT` + 非阻塞 IO；每次 `EPOLLIN`：循环 read，直到 **EAGAIN 或达到单轮读取预算（预算优先）**。
  - **单轮读取预算**：`kMaxReadBytesPerEvent=256KB`（默认写死，可配置）。超过预算则把连接加入 `ReadyQueue` 续处理。
  - **单轮写入预算**：`kMaxWriteBytesPerEvent=256KB`（默认写死，可配置）。写满预算后停止写，注册 `EPOLLOUT` 等待下一轮继续，避免单连接阻塞 EventLoop。
- **参数限制**：**每连接每轮最多处理 5 个完整请求（可配置），用于公平性与尾延迟调优。**

防坑点 1：管线化饥饿 / LT 假死

- **问题**：用户态 Buffer 中已积压完整请求，但为防饥饿单轮只处理 5 个；此时内核 Socket 已读空、不会再触发新的 EPOLLIN。
- **对策**：若已达到上限但 Buffer 仍 `has_full_request()`，必须将该连接加入 `ReadyQueue`，在后续轮次继续解析/处理（严禁依赖下一次 EPOLLIN）。

防坑点 2：DoS 防御与协议边界

- **Header 限制**：`inbuf.size() > 8192` → 返回 **431** 并 close（best-effort 写回）。在寻找 `\r\n\r\n` 过程中，只要 `inbuf.size() > 8192` 就立即 431+close（无需等解析到完整 header）。
- **Body 限制**：默认 `max_body_size = 8MB`（写死默认值，可配置）。若 `Content-Length > max_body_size`，返回 **413** 并 close（best-effort 写回）。
  - **全局预算**：`global_inflight_bytes`（inbuf+outbuf）默认 **512MB**（写死默认值，可配置）；超预算默认 **503 + close（best-effort 写回）**，可配置为 `CLOSE_ONLY`（直接 close 不写回）。
  - **预算更新口径（写死）**：每次追加 `inbuf/outbuf` 前先做 `budget_check(add_bytes)`（判定口径写死：先保证 `global_inflight_bytes<=max_inflight_bytes`，再用 `add_bytes`

`> max_inflight_bytes - global_inflight_bytes` 判超限，避免 `cur+add` 溢出绕过）；仅当实际追加 `inbuf/outbuf` 成功后，才在同一线程内按 `add_bytes` 同步递增 `global_inflight_bytes`；若 `budget_check` 失败，按 overbudget_action 执行（默认 503+close；若配置为 CLOSE_ONLY 则直接 close）。
    - **预算释放口径（写死）**：`inbuf/outbuf` 每次 `consume/shrink/free` 的 delta 必须同步从 `global_inflight_bytes` 递减，使其始终代表当前总量。
- **协议检查**：收到 `Chunked` → **501 + close（best-effort 写回）**；POST 缺 `Content-Length` → **411 + close（best-effort 写回）**。
- **Slowloris 防御**：deadline 按 Content-Length 计算；小包提供 30s 容忍窗口；deadline 上限 120s（但可能被 `body_timeout` 更严格地截断）。
    - **公式**：`body_deadline = now + min(120s, max(30s, Content-Length / 65536Bps))`（注：65536Bps 仅用于 deadline 估算常数，不代表对客户端最低速率承诺）。
    - **判定优先级（写死）**：body 超时判定统一为 `min(body_timeout, body_deadline)`。
- **IP 限流**：`ip_token_bucket`（按 IP 统计 accept 速率）。
    - 必须具备 TTL 过期清理 + 全局最大条目数封顶（超限按 **LRU 淘汰**），防止大量不同 IP 撑爆内存。
    - **默认值（写死）**：`ttl=10min`，`max_entries=100k`；超限淘汰策略 **写死为 LRU**。
    - **TTL 语义（写死）**：`ttl=10min` 按 `last_seen` 滑动过期（超过 TTL 未访问则回收条目）。
    - **Token 口径（写死）**：每次 `accept` 成功消耗 1 token；按 `refill=50 tokens/s` 补充，`burst=200`（桶容量）；不足则按默认 **429 + close（best-effort 写回）** 处理。 新建 IP 条目初始化 `tokens=burst`，`last_refill=now`（写死）。
    - 超限策略：默认 **429 + close（best-effort 写回）**；可配置 `CLOSE_ONLY`（直接 close，不写回）。
- **错误响应口径（收口）**：如选择返回错误响应则 best-effort 写回（能写则写，写不完或 EAGAIN 则 close）；`CLOSE_ONLY` 分支直接 close，不保证客户端收到完整响应。

连接管理

- **连接数封顶（写死）**：`max_conns = 200k`（默认写死，可配置）；计数对象=epoll 管理且 `closing==false` 的连接数（accept 成功 +1；首次置 `closing=true` 时 -1（只允许一次））；accept 后若超限：先置 `closing=true`（触发计数口径 -1），再 `epoll_ctl(DEL)` 并 `close(fd)` 并同时移除 fd→conn 映射/置 fd=-1（映射清理不替代 `epoll_ctl(DEL)` 与 `close(fd)`），随后入 `pending_close_queue` 统一释放，并递增 `conn_reject_total`。
- **生命周期**：关闭连接时不立刻 `delete`；入 `pending_close_queue` 前必须 `epoll_ctl(DEL)` 并 `close(fd)` 并同时移除 fd→conn 映射/置 fd=-1（映射清理不替代 `epoll_ctl(DEL)` 与 `close(fd)`），在本轮 eventloop 末尾统一释放。 关闭流程唯一入口为置 `closing=true`；`epoll_ctl(DEL)`、`close(fd)`、清映射与入 `pending_close_queue` 均由该入口函数统一顺序执行。
    - **防 UAF（写死）**：入 `pending_close_queue` 前必须已标记 `closing=true`，并从 `ReadyQueue`/超时结构摘除；事件分发入口若 `closing==true` 直接跳过该连接任何读写/回调；若仍被取出则检测 `closing` 直接丢弃，避免 UAF/误处理。`closing=true` 只能在连接进入关闭流程的**唯一入口函数**中置位（包括超限拒绝与正常关闭），其他任何路径严禁置位。
- **超时细化**：
    - `header_timeout`（默认 10s）：连接建立后必须在规定时间内发完 Header。
    - `body_timeout`（默认 120s）：Header 接收后必须在 `min(body_timeout, body_deadline)` 内发完 Body。
    - `idle_keepalive_timeout`（默认 60s）：空闲 Keep-Alive 连接的存活时间。

验收标准

1. **复用率**：`requests_total / accept_count > 10` （若 `accept_count==0`：该轮无效，不参与验收） 若出现无效轮（`accept_count==0`），按“自测分层”规则丢弃并补跑至 3 轮有效后再取中位数；不足 3 轮有效则该用例整体无效、不参与验收。
   - **压测脚本**：`wrk -t8 -c100 -d35s --latency` （前 5s warmup；后 30s 为测量窗口，按差分统计；`accept_count==0`：该轮无效，不参与验收（不计算复用率））。
2. **内存稳定性**：1000 并发长连接，warmup 60s 后记录基线 RSS，运行 10 分钟后增量 < 10MB。
   - **RSS 口径（写死）**：RSS 以 `/proc/<pid>/status` 的 `VmRSS` 为准，取 warmup 结束时与 10 分钟结束时差分。
   - **采样规则**：同配置跑 3 轮统一取中位数作为结论；最坏值仅记录，不用于 pass/fail 判定。

☑ DoD（MVP 封板交付清单）

- ☐ `make release && ./scripts/run_demo.sh`：启动网关并能被 `curl`/`wrk` 访问。
- ☐ 基础边界响应可复现：431/413/411/501（best-effort 写回 + close）。
- ☐ `wrk` 压测用例跑通并落盘 3 轮中位数（含 latency samples 与 summary）。
- ☐ `make asan_ubsan` 口径下跑压力 ≥5 分钟无内存告警；关键关闭路径无崩溃。
- ☐ README 说明：单 Reactor 模型、读写预算、ReadyQueue 的触发条件与复现步骤。

---

3) Project 3：简易即时通讯 IM（MVP）

功能定义：基于内存的用户登录、点对点转发。

技术实现

- **存储结构**：`unordered_map<string, SessionEntry{Connection* conn; uint64_t conn_id}>`。
- **线程模型**：`username_map` 的所有访问必须在同一个 EventLoop(IO) 线程内序列化完成。

防坑点（野指针/生命周期）

- **重连误删（写死）**：`close` 回调时，校验/删除使用 `find()`，不使用 `operator[]`；仅当 key 存在且 `conn_id` 匹配才 `erase`，否则不创建条目。
- **Connection 释放**：采用 defer：close 不立刻 delete，入 `pending_close_queue` 前先 `epoll_ctl(DEL)` 并 `close(fd)` 并同时移除 fd→conn 映射/置 fd=-1（映射清理不替代 `epoll_ctl(DEL)` 与 `close(fd)`），在本轮 EventLoop 末尾统一释放（与网关一致）；事件分发入口若 `closing==true` 直接跳过该连接任何读写/回调。
- **转发路径防御**：先比对 map 中保存的 `conn_id`，校验通过后才使用 conn 指针；不通过直接丢弃并可计数 `stale_session_drop_total`。

内存保护

- **背压**：每连接 `outbuf` 上限 **1MB**（按“未发送待写字节数”计算）；超过上限直接断连并记录 `outbuf_overflow_total`。
- **离线消息限制（收口）**：`max_offline_msgs` 与 `max_offline_bytes` **同时生效**：任一达到上限即执行 `DROP_OLDEST`；`offline_drop_total` **每次入队尝试只计一次**；`max_offline_bytes` 口径同 `global_offline_budget`：统计离线 **payload 字节**（不含协议头）；入队成功加，投递成功或丢弃时减。默认 `max_offline_msgs=1000`、`max_offline_bytes=1MB`（per-session，可配置），超限仍按现有 `DROP_OLDEST` 与 `offline_drop_total` 口径执行。

- **全局封顶（默认启用）**：`global_offline_budget_bytes = 512 * 1024 * 1024`、`max_sessions = 100k`（写死默认值，可配置；`max_sessions` 计数对象 = `username_map.size()`，尝试创建新 key 前若 `size>=max_sessions` 则拒绝并计数 `session_reject_total`）；超限时**拒绝新会话**并计数 `session_reject_total`，离线入队按 `DROP_OLDEST` 丢弃并计数 `offline_drop_total`。
  - **global_offline_budget 口径（写死）**：统计离线 **payload 字节**（不含协议头）；入队成功加，投递成功或丢弃时减；同一次入队若触发多个上限，执行 `DROP_OLDEST` **直到所有上限满足**（或队列为空）；`offline_drop_total` **每次入队尝试只计一次。** 若单条离线 `payload_bytes > max_offline_bytes` 或 `payload_bytes > global_offline_budget_bytes`，直接拒绝该条入队并计 `offline_drop_total`，且严禁进入 `DROP_OLDEST` 循环。

验收标准

1. **端到端延迟**：P99 < 50ms
   - **口径**：loopback，消息 256B，1000 连接；warmup **1s**，测量 **10s**；样本量=测量窗口内消息数；P99 按统一 nearest-rank 算法。

☑ DoD（MVP 封板交付清单）

- [ ] `make release && ./scripts/run_demo.sh`：启动 IM；两客户端能登录并互发消息。
- [ ] `conn_id` 校验 + defer 释放路径可复现（快速重连不误删）。
- [ ] 背压生效：`outbuf` 超限断连并计数（可写一个脚本复现）。
- [ ] P99 口径压测跑通并落盘 3 轮中位数（含 raw samples 与 summary）。
- [ ] README 说明：会话表、生命周期与离线消息边界、如何复现 P99 用例。

🚀 阶段二：工业级迭代（杀回去）

1) Project 1：超低延迟关键路径 lock-free 日志库（不宣称 wait-free）（Pro）

Pro 开始条件（闸门，写死）：必须满足以下条件才允许进入 Pro：

- MVP DoD 全绿（交付清单全部打勾）。
- 压力用例至少 3 轮有效结果已落盘（取中位数），可复现。
- 具备一段可复述的 Evolution Story：踩坑 → 定位 → 修复 → 指标变化（有证据）。

核心升级：MPSC RingBuffer + Crash Safety

- **MPSC 写入正确性**
  - **CAS 判满**：Reserve 使用 CAS 循环：在推进 `write_index` 前判定剩余空间；空间不足则不推进，直接 drop。
  - **内存序（写死）**：生产者提交记录使用 `release` 发布（写完 payload/元数据后再发布 state/seq），消费者读取使用 `acquire` 获取，确保不会读到半写数据。
  - **一致性**：drop 不能让消费者卡在 `WRITING`；必要时发布可消费的 `PADDING/SKIP` 记录推进读指针，尽量让消费者可持续前进（best-effort）。
- **消费者防死锁**
  - 遇到 `WRITING` 自旋等待，但必须受 `exit_flag` 或超时阈值保护（触发则 best-effort shutdown）。**默认值（写死）**：`spin_timeout=100ms`（可配置）；超时则置 `exit_flag=true` 并触发 best-effort shutdown（计数）。

- **Crash Safety（严谨口径）**
  - **承诺边界**：Crash Safety 为 best-effort；不保证 SIGSEGV/强杀后一定完成刷盘。
  - **信号处理**：仅执行 async-signal-safe 操作（写 self-pipe 或设置 `volatile` 标志）。

---

2) Project 2：高性能网关（Pro）

Pro 开始条件（闸门，写死）：必须满足以下条件才允许进入 Pro：

- MVP DoD 全绿（交付清单全部打勾）。
- 压力用例至少 3 轮有效结果已落盘（取中位数），可复现。
- 具备一段可复述的 Evolution Story：踩坑 → 定位 → 修复 → 指标变化（有证据）。

核心升级：时间轮 + 尽量零拷贝

- **零拷贝边界**：静态文件使用 `sendfile`；动态响应使用 `writev` 减少拼接拷贝（不承诺全链路零拷贝）。
- **时间轮所有权**：时间轮存 `weak_ptr<Connection>`；`EventLoop` 强持有 `shared_ptr`。
- **环引用边界**：回调/定时器仅捕获 `weak_ptr`，使用时 `lock()`。

---

3) Project 3：高可靠 IM（Pro）

Pro 开始条件（闸门，写死）：必须满足以下条件才允许进入 Pro：

- MVP DoD 全绿（交付清单全部打勾）。
- 压力用例至少 3 轮有效结果已落盘（取中位数），可复现。
- 具备一段可复述的 Evolution Story：踩坑 → 定位 → 修复 → 指标变化（有证据）。

核心升级：ACK + 序列号 + 去重

- **QoS 一致性**：至少一次送达 (At-Least-Once) + 业务层去重（窗口内去重实现幂等、降低重复可见性；不宣称 Exactly-Once 语义）。
  - **跨重启语义**：窗口内为内存去重，不跨重启；跨重启需依赖业务幂等或可选的持久化去重。
- **有序性（收口）**
  - **Session Seq**：服务端分配会话内单调递增 seq；接收端可按 seq 重排/检测乱序（至少一次语义下可能出现重复/乱序可见）。
  - **Snowflake**：仅用于去重；需处理时钟回退（逻辑时钟模式）。
- **去重表边界（收口）**：超过 10min 按 `last_seen` 滑动过期（now-last_seen>10min 则回收）；同时容量封顶 1,000,000 entries（计数对象=去重 entry 数），超限按 LRU 淘汰最老条目（唯一策略）；超限时在本次插入前先淘汰至低于上限后再插入。
  - **全局封顶**：`dedup_budget_mb = 512MB`（写死默认值，可配置）；超限按会话 LRU 淘汰最老会话（唯一策略）；超限时在本次创建/插入前先淘汰至低于上限后再执行。
  - **dedup_budget 计量口径（写死）**：每个 dedup entry 逻辑记账为 `key.size() + 32B` 字节；插入成功加，过期/淘汰时减，用于近似表示去重表的逻辑记账占用（best-effort）；实际 RSS 可能略高。
- **重试策略**：指数退避 + Jitter（50ms 起，上限 2s）。超过 N 次（默认 N=5，可配置）后固定为“失败上报 + 指标计数”，不再重试。

🎯 20k Linux C++ 服务端面试“主打点”（60 秒版本）

- **我做的是“可落地的高性能服务端”，不是 PPT**：所有优化必须能用统一 Benchmark 口径复现，P99 算法与统计窗口写死，并保留 raw samples 复算。

- **我能把系统“关进笼子”**：OOM/DoS/超时/限流都有默认封口与失败语义（拒绝/写回/close），“最好努力”但边界明确。
- **我能讲清并发与生命周期**：stop/shutdown 语义写死；关闭连接 defer 释放、防 UAF；关键路径尽量单线程串行化全局结构，跨线程只留原子计数。
- **我不靠口嗨证明稳定**：release + asan_ubsan + tsan 三套口径自测，压力用例三轮取中位数，证据落盘可审计。
- **我能定位瓶颈**：每个项目都保留关键指标（例如 enqueue_ns_p99 / write_bytes_per_sec / RSS 增量），能区分“锁争用 vs IO vs 协议边界”。

💡 面试防御逻辑（Evolution Story）

> “MVP 阶段我遇到了三个工程硬坑：
> 1) **管线化饥饿**：Epoll LT 下 Buffer 有数据但内核不触发事件导致假死，所以我加了 **ReadyQueue**。
> 2) **大包攻击**：没限制 Body 大小导致内存爆，所以我加了 **8MB 上限** 和 **413 拒绝**。
> 3) **重连误删**：快速重连时 Close 回调删错了 Map 条目，所以我用 `find()+conn_id` 做一致性校验并 defer 释放。
> 这些坑都是我在压测和调试时踩出来的，并用可复现数据验证修复效果。”

🛠 开发者：高频实现陷阱（逻辑一致性检查）

> **目的**：确保代码实现与 Metrics 统计口径 严格对齐，防御“统计造假”质疑。

1. **资源计数的“守恒定律”**
   - **原则**：所有 `budget` 或 `current_bytes` 计数器，必须保证 **“在哪加，就在哪减；加多少，就减多少”**。
   - **检查点**：
     - **Project 1**：`current_bytes` 在 `Log()` 持锁入队时增加，必须在后台线程持锁 `pop` 后立即减少。严禁出现“只管入队，不管释放”的逻辑断层。
     - **Project 3**：离线消息的 payload 字节数在入队时增加，在“投递成功”或“DROP_OLDEST 丢弃”时必须严格减去。
2. **空样本的“物理剔除”**
   - **原则**：严禁将“未发生操作的 0 延迟”计入 P99 统计样本。
   - **检查点**：在计算 P99 时，如果该测量窗口内 `requests_total` 或 `enqueue_total` 的增量为 0（即无流量），必须在 `summary.md` 中将延迟标记为 `N/A`，**严禁记录为 0ms**，否则会拉低 P99 造成数据造假嫌疑。
3. **连接计数的“唯一出口”补充**
   - **细节**：在 Project 2/3 中，`closing=true` 是所有连接销毁的**唯一入口**。
   - **检查点**：即使在 `accept` 后立即发现超限，也必须调用这个唯一入口函数来置位 `closing=true` 并执行关闭流程。严禁直接 `close(fd)` 而不触发计数器逻辑，否则 `reject_total` 和 `conns_current` 将永远对不上账。
4. **Stop 竞态的“二次检查”（Project 1）**
   - **原则**：`Stop()` 生效后，**新的 `Log()` 调用不得成功入队**。
   - **检查点**：`Log()` 必须在**入锁前**与**入锁后**各检查一次 `m_stop`；若任一处为 true 则拒绝并计入 `dropped_stop_total`，确保存停过程口径唯一。
5. **时间口径只用单调时钟**
   - **原则**：延迟样本与 P99 统计必须使用单调时钟，避免 NTP/手动调时污染。
   - **检查点**：采样统一使用 `steady_clock` / `clock_gettime(CLOCK_MONOTONIC)`，严禁 `system_clock` 参与延迟统计。

6. **差分统计的“同源采样”**
    - **原则**：差分的基线与终点必须同源，同一套计数器、同一输出来源，避免不可复现。
    - **检查点**：凡写“按差分统计”的指标，基线与终点均从 `metrics.log` 同一 key 读出（或等价同源采样），禁止混用不同来源。
7. **容器隐式插入的“审计禁区”**
    - **原则**：读/删路径严禁 `operator[]` 触发隐式创建，避免计数与内存悄然漂移。
    - **检查点**：Map/桶结构（session、限流桶、LRU 等）在读取/删除时必须使用 `find()`；`operator[]` 仅允许用于明确的创建/写入路径。

🌓 审查人说明

前面是“法律”（What to do）：规定了必须怎么做。

后面是“执法手册”（How to verify）：规定了怎么检查有没有做对。